ICS 441 · LAB 1

VulnFeed Security Risks

Six vulnerabilities found in the "VulnFeed" social media app — explained simply, with fixes anyone can understand.



CRITICAL

Open Storage Bucket

1

The Problem

The place where all user photos are stored is unlocked. Anyone on the internet can open it and download every single photo — no password, no login, nothing.

Think of it like...

A storage locker at a facility that anyone walking by can open and take stuff from, because there's no lock on the door at all.

The Fix

- Lock the bucket — make it private
- Only the app itself can open it
- Give temporary, one-time keys when a user needs to see their own photo

CRITICAL

💉 SQL Injection in Search

2

😨 The Problem

The search box on the homepage doesn't check what you type. Attackers typed a special trick and got the site to hand over every username, email, and password in the database.

🤔 Think of it like...

A mailbox that's supposed to accept a simple letter, but if you slip in a sneaky note, the mail sorter gets tricked into handing over everyone else's mail too.

✅ The Fix

- Never paste user input straight into a database question
- Use "parameterized queries" — a safe way to ask questions
- Double-check anything a user types before using it

CRITICAL

🔑 Broken Access Control

3

😩 The Problem

You can look at anyone's private posts just by changing a number in the web address — even if their account is set to private. The app never checks if you're actually allowed to see it.

🏨 Think of it like...

A hotel where your keycard should only open your room, but if you just tell the front desk "I'm in room 205," they let you walk right in — no ID check.

✅ The Fix

- Always check: does this person actually own this data?
- Never trust an ID number typed in the URL
- Verify ownership on the server, every single time

HIGH

🗝 Hardcoded API Keys

4

😨 The Problem

A secret key that lets the app send emails/maps requests is written in plain sight in the website's public code. Someone already found it and ran up a huge bill using it.

🏦 Think of it like...

Writing your credit card number on a sticky note and taping it to your front door — anyone walking past can see it and start spending your money.

✅ The Fix

- Remove the key from public code immediately
- Move it to the backend, hidden from users
- Cancel the leaked key and issue a new one

HIGH

📱 Missing MFA (2-Step Login)

5

😩 The Problem

Logging in only needs a username and password. If someone steals or guesses your password, they get full access — there's no second check to stop them.

🚪 Think of it like...

A house with only one lock on the front door. Once someone copies that one key, they're inside — there's no second door or alarm to slow them down.

✅ The Fix

- Add a second login step (code sent to phone or email)
- Require it for admin accounts at minimum
- Offer it to all users for better protection

MEDIUM

🔓 Weak Password Policy

6

😩 The Problem

The site only requires a 5-character password with no rules on complexity. Short, simple passwords are easy for attackers to guess or crack by trying thousands of options.

🔗 Think of it like...

Locking your front door with a flimsy twist-tie instead of a real deadbolt. It technically "locks," but it won't stop anyone determined to get in.

✔ The Fix

- Require longer passwords (12+ characters)
- Block common/breached passwords
- Add limits on repeated failed login attempts

📊 Priority Ranking (Highest → Lowest Risk)

Rank	Vulnerability	Why it's ranked here
1	💉 SQL Injection in Search	Directly and easily steals every password in the database — no special access needed
2	🔑 Broken Access Control	Attacker can view anyone's private data just by changing a number — trivial to exploit
3	🪣 Open Storage Bucket	All user files exposed publicly, but "only" media, not passwords
4	🗝️ Hardcoded API Keys	Already being abused financially, but doesn't expose user data directly
5	📱 Missing MFA	Serious, but requires the attacker to already have a password (e.g. via phishing)
6	🔓 Weak Password Policy	Makes attacks easier, but still needs real effort (brute-force) to actually get in

🛡 The Big Picture

The most dangerous risks are the ones an attacker can exploit **directly, immediately, and without extra effort** (SQL injection, broken access control, open storage). The lower-ranked risks still matter — they just need an extra step, like a phished password, before they can be used against you.